October 2026

Who Wrote It? Identifying LLMs from Their Responses

CMPSC 448 MIDTERM PROJECT

PENN STATE UNIVERSITY

Table of Contents

1. Introduction and Problem Definition
1.1 Motivation: the LLM authorship problem
1.2 Task formulation (multi-class classification over LLM families)
1.3 Research questions addressed (RQ1–RQ4)
1.4 Summary of main findings

2. Dataset Curation
2.1 Data sources and collection procedure
2.2 LLM families included
2.3 Prompt and task categories
2.4 Cleaning and preprocessing
2.5 Dataset statistics and class balance
2.6 Train / validation / test splits
2.7 Data rights, privacy, and ethical considerations
2.8 Dataset design

3. Models
3.1 Text representation (tokenization, vocabulary, embeddings)
3.2 CNN architecture
3.3 RNN (LSTM) architecture

4. Training Details
4.1 Hyperparameters and how they were selected
4.2 Optimization and regularization
4.3 Evaluation metrics
4.4 Reproducibility

5. Results
5.1 RQ1: Can We Identify Which LLM Generated a Response?
5.2 RQ2: Does the User Prompt Help Identify the LLM?
5.3 RQ3: How Well Do the Models Generalize an Unseen Domain?
5.4 RQ4: What Linguistic and Structural Characteristics Help Distinguish LLM Families?

6. In-Depth Analysis
6.1 CNN vs. RNN/LSTM Comparison
6.2 Domain Generalization Analysis
6.3 Linguistic and Structural Signal Analysis

6.4 Error and Confusion Analysis
6.5 Ablation Analysis

7. Lessons and Experience
7.1 What worked and what didn't
7.2 Challenges and how they were resolved
7.3 What we would do differently

8. Conclusion and Limitations
8.1 Conclusion
8.2 Limitations
8.3 Future Work

References

Appendix
A. Team Member Contributions
B. Statement on LLM Assistance

1. Introduction and Problem Definition

1.1 Motivation: the LLM authorship problem

Large language models from different model families often produce responses that differ in subtle but potentially systematic ways. Even when multiple models answer the same or similar prompts, their responses may vary in wording, response length, organization, formatting, vocabulary, and level of detail. These differences raise an interesting question: **can the model family that generated a response be identified from the response itself?**

In this project, we study LLM authorship identification as a text-classification problem. Specifically, we investigate whether neural classifiers can distinguish responses produced by three major LLM families: **GPT, Claude, and Gemini**.

A major challenge is ensuring that classification performance reflects meaningful patterns in model outputs rather than simple dataset shortcuts. For example, a classifier could obtain artificially high performance if responses explicitly identify the generating model, if identical prompts appear in both the training and test sets, or if one model family is associated much more strongly with a particular task domain. Therefore, a significant part of this project focuses on careful dataset curation, balancing, leakage prevention, and controlled evaluation.

In addition to comparing CNN and RNN-based classifiers, we investigate whether model identification changes depending on whether the classifier receives the prompt, the response, or both. We also study whether learned model signatures generalize across different task domains and analyze linguistic and structural characteristics that may distinguish the three LLM families.

1.2 Task formulation (multi-class classification over LLM families)

We formulate the problem as a **three-class text classification task**.

For each example, the target label is one of three LLM families:

- GPT

- Claude
- Gemini

The input to the classifier depends on the research question being evaluated.

For **RQ1**, the classifier receives only the LLM-generated response and predicts the corresponding model of family.

For **RQ2**, we compare three input settings:

1. prompt only,
2. response only,
3. prompt and response combined.

Two neural text-classification architectures are evaluated: a **Convolutional Neural Network (CNN)** and a **Recurrent Neural Network based on LSTM**.

For additional analyses, domain information and response-level linguistic characteristics are used to study generalization and model-specific writing patterns.

1.3 Research questions addressed (RQ1, RQ2, RQ3, and RQ4)

This project addresses the following four research questions:

RQ1: LLM Identification from Output

Can a CNN or RNN classifier determine whether a response was generated by GPT, Claude, or Gemini using only the LLM-generated output?

RQ2: Effect of Input Information

How does classification performance change when the classifier receives:

- the prompt only,
- the response only, or
- the prompt and response together?

This experiment also helps determine whether the prompts themselves contain unintended signals correlated with the model family.

RQ3: Cross-Domain Generalization

Does a classifier trained on responses from some task domains continue to identify the LLM family when evaluated on a different domain?

This experiment examines whether the classifier learns more general model-specific patterns or mainly relies on patterns associated with particular types of tasks.

RQ4: Linguistic and Structural Characteristics
What characteristics of GPT, Claude, and Gemini responses differ across model families?

We examine characteristics such as response length, lexical usage, formatting patterns, Markdown structure, headings, and list usage to better understand what signals may contribute to LLM identification.

1.4 Summary of main findings

The experiments show that LLM families can be distinguished from their generated responses within the scope of our dataset. For RQ1, the CNN achieved **91.96% accuracy and 92.01% Macro-F1**, substantially outperforming the RNN/LSTM, which achieved **76.00% accuracy and 75.94% Macro-F1**. RQ2 showed that most of the identifying signal comes from the generated response rather than the user prompt: prompt-only classification performed close to chance for both architectures, while response-based classification was substantially stronger.

The cross-domain experiments in RQ3 showed that the learned signals generalize across domains to some extent, although performance depended on both the architecture and the held-out domain. The CNN was generally more robust across domain shifts, while the RNN/LSTM experienced its largest performance decrease when Code was excluded from training.

Finally, the RQ4 analyses suggest that the classifiers rely on a combination of lexical and structural characteristics. Response length, formatting, punctuation, recurring local phrases, and response organization all provided useful signals. Removing punctuation and formatting notably reduced RNN/LSTM performance, indicating that structural writing patterns contribute meaningfully to LLM-family identification.

Overall, the CNN was the stronger and more consistent classifier in our experiments. However, the results should be interpreted as evidence of **detectable family-associated patterns within the selected dataset and model versions**, rather than as proof of universal or permanent LLM fingerprints.

2. Dataset Curation

2.1 Data sources and collection procedure

The dataset used in this project was derived from the publicly available **LMArena Arena Human Preference 140K dataset** (LMArena, 2025). The loaded dataset contained **135,634 interaction records**.

The original Arena data is organized around model comparison interactions. A record may contain a user's conversation together with responses generated by two different language models. Because our task requires individual examples of the form:

(LLM family, prompt, response),

We transformed the original comparison-oriented records into individual response-level examples.

Before constructing the final dataset, we inspected the structure of the conversation fields, available model names, conversation lengths, language distribution, and task metadata. We restricted the primary dataset to **English single-turn interactions**, where the conversation consists of one user prompt followed by one assistant response. This decision simplified the classification setting and reduced variation caused by multi-turn conversational history.

Each extracted response was mapped from its exact model identifier to one of three model families:

- GPT,
- Claude,
- Gemini.

The resulting candidate pool was then cleaned, inspected for leakage, controlled for model-version composition, and sampled to create the final balanced dataset.

2.2 LLM families included

The project focuses on three LLM families: **GPT, Claude, and Gemini**.

Rather than using every available version of each family in the source dataset, we selected three relatively well-represented model versions from each family. This was done to reduce the possibility that one family would be represented primarily by a large number of examples from a single model version.

The selected models were:

Family	Model versions included
GPT	chatgpt-4o-latest-20250326, gpt-4.1-2025-04-14, gpt-4.1-mini-2025-04-14
Claude	claude-opus-4-20250514, claude-sonnet-4-20250514, claude-3-7-sonnet-20250219-thinking-32k
Gemini	gemini-2.5-flash, gemini-2.5-pro, gemini-2.0-flash-001

This selection produced a large candidate pool while maintaining diversity within each model family.

It is important to note that the objective of the project is **family-level classification**, not identification of an exact model version. Therefore, all selected versions belonging to the same family share the same family label.

2.3 Prompt and task categories

To support both balanced sampling and cross-domain analysis in RQ3, the task metadata provided by the Arena dataset was used to organize examples into broad domains.

The final dataset contains four primary domain categories:

- **General**

- **Code**
- **Math**
- **Creative Writing**

Examples marked by more than one of the code, math, or creative-writing indicators were treated as mixed-domain examples during dataset construction and were not included in the final controlled sample.

The general category represents examples that were not marked as code, math, or creative writing according to the available Arena metadata. Therefore, it should not be interpreted as a formally defined universal task category, but rather as the remaining non-tagged portion of the selected data.

Controlling the domain distribution was important because otherwise a classifier might learn that a particular LLM appears more frequently in a particular type of task instead of learning characteristics of the model's responses.

2.4 Cleaning and preprocessing

Several cleaning and quality-control steps were applied before constructing the final dataset.

Empty responses

We first checked for missing and empty prompt and response fields. No empty prompts were found, but **23 empty responses** were identified and removed.

Exact duplicates

We checked for exact duplicate examples based on the model family, exact model, prompt, and response. A total of **47 duplicate copies** were removed while retaining one copy of each duplicated example.

Direct model-identity leakage

Responses were inspected for cases in which the model directly revealed its own identity, such as statements similar to:

“I am Claude…”

Such examples would make the classification task artificially easy because the answer would directly contain the target label.

A conservative family-specific filter was therefore used to remove obvious self-identification cases. This process removed **153 responses** containing clear direct identity leakage.

The filter was intentionally conservative. We did not remove every response that happened to mention words such as “GPT,” “Claude,” or “Gemini,” because these terms can legitimately occur as part of the user's requested topic.

Short responses

Very short responses were inspected rather than automatically removed. Some short responses were legitimate answers to prompts requesting brief outputs. Because these examples are valid interactions, short responses were retained.

Preservation of text structure

We deliberately avoided aggressive normalization of the response text.

In particular, we preserved:

- capitalization,
- punctuation,
- paragraph structure,
- newlines,
- Markdown,
- headings,
- lists,
- code formatting.

These characteristics may themselves represent useful stylistic or structural signals for distinguishing LLM families and are particularly relevant to RQ4.

2.5 Dataset statistics and class balance

After cleaning and model-version selection, we constructed a controlled final dataset containing **6,000 examples**.

The three target classes were exactly balanced:

LLM Family	Number of Examples
GPT	2,000
Claude	2,000
Gemini	2,000
Total	**6,000**

Balancing the classes prevents overall accuracy from being dominated by the most frequent LLM family.

The domain distribution was also controlled within each family.

For each LLM family, we sampled:

Domain	Samples per Family
General	800
Code	500
Math	350
Creative Writing	350
Total	**2,000**

Therefore, across all three families, the final dataset contains:

Domain	Total Samples
General	2,400
Code	1,500
Math	1,050
Creative Writing	1,050

Total	**6,000**

This controlled sampling strategy reduces the possibility that model-family predictions are driven primarily by differences in task-domain frequency.

However, the final dataset is therefore a **controlled experimental sample**, rather than a representation of the natural frequency of task domains in the original Arena dataset. This distinction is important when interpreting the results.

2.6 Train / validation / test splits

The 6,000 curated examples were divided into training, validation, and test sets.

The resulting split sizes were:

Split	Number of Examples
Training	4,188
Validation	916
Test	896
Total	**6,000**

The intended proportions were approximately **70% training, 15% validation, and 15% test**.

A key concern during splitting was **prompt leakage**.

The same exact prompt can appear in multiple Arena interactions and may be answered by different model families. If responses to the same prompt appeared in both training and test sets, the classifier might partially benefit from having already seen the prompt during training.

To prevent this, two identifiers were retained:

- group_id: the original Arena interaction identifier,
- prompt_group_id: a SHA-1-based identifier created from the exact prompt text.

The dataset was split using prompt_group_id as the grouping variable. As a result, all examples corresponding to the same exact prompt were assigned to the same dataset partition.

The final leakage checks confirmed:

- training–validation prompt overlap: 0
- training–test prompt overlap: 0
- validation–test prompt overlap: 0

Because entire prompt groups were kept together, the final split sizes are not exactly 70/15/15. Preventing prompt leakage was prioritized over obtaining mathematically exact split proportions.

The same fixed train, validation, and test partitions were provided to both the CNN and RNN teams to ensure a fair comparison between classifiers.

2.7 Data rights, privacy, and ethical considerations

The project uses the publicly available **LMArena Arena Human Preference 140K** dataset rather than collecting new model interactions specifically for this study. According to the official dataset card, **user prompts are licensed under the Creative Commons Attribution 4.0 (CC BY 4.0) license, while model-generated outputs are governed by the terms of use of their respective model providers.**

Only information needed for the classification experiments was retained in the curated dataset. No attempt was made to identify individual users or connect prompts to real-world identities. Because the dataset contains user-generated conversational text, it was used only for the intended educational and research analysis, and individual users were not profiled or analyzed.

2.8 Dataset Design

Overall, the curation process was designed to create a controlled experimental dataset for studying LLM-family identification. Class balancing supports fair comparison in RQ1 and RQ2; domain metadata enables cross-domain experiments in RQ3, and preservation of response-level structural and linguistic characteristics enables the analysis in RQ4. The removal of direct identity leakage and the use of prompt-grouped train/validation/test splits were intended to reduce obvious shortcuts and provide a more meaningful evaluation of model-family classification.

3. Models

3.1 Text representation (tokenization, vocabulary, embeddings)

Both classifiers operate on tokenized versions of the original text while preserving structural information that may be useful for LLM-family identification. The tokenizer separates word sequences, newlines, and punctuation using the regular expression \w+|\n|[^\w\s]. Capitalization, punctuation, Markdown symbols, and newline information are therefore retained rather than removed through aggressive preprocessing.

The vocabulary is constructed using the **training split only**. Tokens appearing at least twice in the training data are included in the vocabulary, while less frequent or previously unseen tokens are mapped to an <unk> identifier. A separate <pad> identifier is used when sequences need to be padded.

For response-only experiments, each response is limited to the first **512 tokens**. For the RQ2 prompt-only condition, the input is limited to **128 prompt tokens**. For the combined condition, up to **128 prompt tokens** are followed by a separator marker and up to **512 response tokens**. This standardized representation allows the CNN and RNN/LSTM to receive comparable input across the RQ2 experiments.

Each token is converted to its corresponding vocabulary ID before being passed to an embedding layer. Shorter sequences are padded with zeros to create equal-length tensors within each experiment. Padding is handled explicitly by both architectures so that artificial padding positions do not become meaningful evidence for classification.

Training examples are processed in batches of **64**. Training data are shuffled during training, while validation and test data are evaluated without shuffling.

3.2 CNN architecture

The CNN classifier follows the general text-classification approach of **Kim (2014)**. The model begins with an embedding layer that converts each token ID into a learned **200-dimensional embedding vector**. These embeddings allow the model to learn useful numerical representations of tokens directly during training.

The model then applies three groups of one-dimensional convolutional filters with kernel widths of **3, 4, and 5 tokens**. Each kernel width contains **100 filters**, giving the model the ability to detect different local combinations of words, punctuation, and formatting symbols.

A ReLU activation function is applied to the convolution outputs, setting negative activations to zero while retaining positive activations. Max pooling is then used to retain the strongest activation produced by each filter across the response. This allows each filter to indicate whether a learned local pattern occurs anywhere in the input.

Because there are 100 filters for each of the three kernel sizes, the pooled outputs form a **300-dimensional feature representation**. Padding-aware masking is used so that convolution windows consisting primarily of padded positions do not incorrectly influence the pooled representation.

A dropout layer with probability of **0.5** is applied to the feature representation during training to reduce overfitting. The final fully connected layer converts the resulting features into three output scores corresponding to **GPT, Claude, and Gemini**. The class with the highest output score is selected as the predicted LLM family.

3.3 RNN (LSTM) architecture

The recurrent classifier uses a **Long Short-Term Memory (LSTM)** network, originally introduced by **Hochreiter and Schmidhuber (1997)**, to predict whether a response was generated by GPT, Claude, or Gemini.

As with the CNN, input tokens are converted to learned **200-dimensional embedding vectors**. The sequence of embeddings is then processed by a single-layer LSTM containing **128 hidden units**.

Because responses have different actual lengths, the model computes the number of non-padding tokens in each input. PyTorch's packed-sequence mechanism is then used so that the LSTM processes only the meaningful portion of each sequence rather than treating padding as part of the response.

The final LSTM hidden state is used as the representation of the input sequence. A dropout layer with probability of **0.5** is applied before a fully connected linear layer produces three output scores corresponding to GPT, Claude, and Gemini. The class with the highest score is selected as the final prediction.

The LSTM was trained for **10 epochs** using the Adam optimizer with a learning rate of **0.001**, cross-entropy loss, and a batch size of **64**. After every epoch, performance was evaluated on the validation set. The model state with the highest validation accuracy was retained and used for final test evaluation.

4. Training Details

4.1 Hyperparameters and how they were selected

The project used fixed hyperparameter configurations rather than conducting a systematic hyperparameter search. Therefore, these values are treated as consistent experimental settings rather than claimed to be optimal. Keeping the major training settings consistent also supported a clearer comparison between the CNN and RNN/LSTM architectures.

Table 1. Training settings used in the final implementations

Setting	CNN	RNN/LSTM

Embedding dimension	200	200
Feature extractor	100 filters each at widths 3, 4, 5	1-layer LSTM, 128 hidden units
Response-only limit	512 tokens	512 tokens
Prompt-only limit	128 tokens	128 tokens
Prompt + Response limit	128 prompt + separator + 512 responses	128 prompt + separator + 512 responses
Batch size	64	64
Training epochs	10	10
Optimizer	Adam	Adam
Learning rate	0.001	0.001
Dropout	0.5	0.5
Gradient-norm clipping	1.0	Not implemented
Random seed	42	42
Model-selection criterion	Validation accuracy	Validation accuracy

4.2 Optimization and regularization

Both models were trained using the **Adam optimizer** with a learning rate of **0.001** and cross-entropy loss for three-class classification. Training was performed for a maximum of **10 epochs**.

After each epoch, the model was evaluated on the validation set. The parameters corresponding to the highest validation accuracy were saved, and this best validation

checkpoint was restored before final evaluation on the held-out test set. This prevented the final test result from being based simply on the last training epoch.

Both architectures used **dropout with probability 0.5** as a regularization technique to reduce overfitting. The CNN additionally applied **gradient-norm clipping at 1.0** during training. Gradient clipping was not used in the final RNN/LSTM implementation.

No systematic hyperparameter optimization or learning-rate scheduling was performed, so differences in performance should be interpreted within the fixed configurations used in this project.

4.3 Evaluation metrics

Performance was evaluated using **Accuracy, Macro Precision, Macro Recall, Macro-F1, and confusion matrices**.

Accuracy measures the fraction of test examples assigned to the correct LLM family. Macro Precision, Macro Recall, and Macro-F1 are computed independently for GPT, Claude, and Gemini and then averaged across the three classes, giving each family equal weight despite small differences in the number of test examples.

Macro-F1 was particularly useful for comparing the CNN and RNN/LSTM because it incorporates both precision and recall while treating each LLM family equally.

Confusion matrices were also used to examine class-specific errors. Rows represent the true LLM family and columns represent the predicted family, making it possible to identify which families were most frequently confused with one another.

Validation performance was used for model selection, while the fixed test split was reserved for final evaluation.

4.4 Reproducibility

The project was implemented in Python using PyTorch, pandas, NumPy, PyArrow, and scikit-learn. To support reproducibility, the CNN and RNN/LSTM experiments used the same fixed training, validation, and test splits. Vocabulary construction used training data only, and random seeds were set to 42 in the final implementations.

Both architectures used the same class-label mapping and core evaluation metrics. Validation accuracy was used to select the best model checkpoint, while the fixed test set was reserved for final evaluation.

5. Results

5.1 RQ1: Can We Identify Which LLM Generated a Response?

RQ1 investigates whether the source LLM family can be identified using only the model-generated response. The CNN and RNN/LSTM classifiers were evaluated on the same fixed held-out test set containing 896 responses from GPT, Claude, and Gemini. Table 1 summarizes the final performance of both architectures.

Table 2. CNN and RNN performance for RQ1 using response-only input

Model	Accuracy	Macro Precision	Macro Recall	Macro-F1
CNN	0.9196	0.9196	0.9207	0.9201
RNN/LSTM	0.7600	0.7600	0.7593	0.7594

The results show that both architectures were able to identify the generating LLM family from its response, although the CNN performed substantially better than the RNN/LSTM. The CNN achieved an accuracy of **91.96%** and a Macro-F1 score of **92.01%**, compared with **76.00%** accuracy and **75.94%** Macro-F1 for the RNN/LSTM. The CNN therefore exceeded the RNN/LSTM by approximately **16 percentage points in accuracy** and **16.1 percentage points in Macro-F1**.

The similarity between accuracy, macro precision, macro recall, and Macro-F1 for each architecture also indicates that the overall performance was relatively balanced across the three LLM families rather than being driven primarily by one class.

The CNN confusion matrix further demonstrates strong classification performance across GPT, Claude, and Gemini. Of the 273 GPT responses in the test set, **259 were correctly**

classified as GPT. For Claude, **266 of 295 responses** were correctly classified, while **299 of 328 Gemini responses** were correctly identified. The RNN/LSTM also learned useful family-specific signals but produced substantially more cross-family classification errors.

These findings provide evidence that the generated responses in our dataset contain detectable patterns that can help distinguish between GPT, Claude, and Gemini. Such patterns may include differences in lexical choices, local phrasing, formatting, response structure, and other stylistic characteristics. The stronger CNN performance suggests that local textual patterns captured by convolutional filters were particularly informative for this classification task compared with the sequential representation learned by the LSTM.

However, these results should **not** be interpreted as evidence of universal or permanent fingerprints for these LLM families. The experiment uses a curated dataset containing specific model versions and a controlled mixture of domains. Classification performance may therefore be influenced by characteristics such as response length, formatting, domain distribution, or other dataset-specific signals. These potential factors are examined further in RQ3 and RQ4.

CNN Confusion Matrix for RQ1

Actual\Predicted	GPT	Claude	Gemini
GPT	259	9	5
Claude	9	266	20
Gemini	12	17	299

RNN/LSTM Confusion Matrix for RQ1

Actual\Predicted	GPT	Claude	Gemini
GPT	208	31	34
Claude	32	217	46
Gemini	41	31	256

5.2 RQ2: Does the User Prompt Help Identify the LLM?

RQ2 examines whether the user prompt provides useful information for identifying the LLM family. For both CNN and RNN/LSTM, three input settings were evaluated using the same fixed train, validation, and test splits:

1. **Prompt only** – the classifier receives only the user's input prompt.
2. **Response only** – the classifier receives only the generated LLM response.
3. **Prompt + Response** – the classifier receives both the user prompt and the generated response.

This comparison helps determine whether classification performance comes primarily from characteristics of the generated response or whether the original user prompt also contributes useful predictive information.

Table 3. CNN and RNN/LSTM performance for the three RQ2 input settings

Model	Input Setting	Accuracy	Macro Precision	Macro Recall	Macro-F1
CNN	Prompt only	0.3270	0.3498	0.3365	0.3144
CNN	Response only	0.9275	0.9298	0.9286	0.9281
CNN	Prompt + Response	0.9252	0.9287	0.9265	0.9260
RNN/LSTM	Prompt only	0.3248	0.3195	0.3181	0.3117
RNN/LSTM	Response only	0.7734	0.7728	0.7735	0.7727
RNN/LSTM	Prompt + Response	0.7746	0.7750	0.7737	0.7737

The prompt-only condition performed poorly for both architectures. The CNN achieved only **32.70% accuracy** and a **31.44% Macro-F1**, while the RNN/LSTM achieved **32.48% accuracy** and a **31.17% Macro-F1**. These values are close to the approximately one-third level expected when choosing among three classes without strong predictive information. This suggests that, under our experimental setup, the user prompt alone contains little

useful information for determining whether GPT, Claude, or Gemini generated the response.

Performance increased substantially when the generated response was used as the input. The CNN response-only classifier achieved **92.75% accuracy** and **92.81% Macro-F1**, representing an improvement of approximately **60 percentage points in accuracy** compared with prompt-only classification. The RNN/LSTM showed the same general pattern, increasing from **32.48%** prompt-only accuracy to **77.34%** response-only accuracy.

Adding the prompt to the response did not meaningfully improve performance. For the CNN, accuracy slightly decreased from **92.75%** with response-only input to **92.52%** with prompt + response, while Macro-F1 decreased from **92.81%** to **92.60%**. For the RNN/LSTM, adding the prompt produced only a very small improvement, from **77.34%** to **77.46%** accuracy and from **77.27%** to **77.37%** Macro-F1.

These results indicate that the majority of the predictive signal used to identify the LLM family comes from the **generated response rather than the user's prompt**. The prompt by itself provides little discriminative information, and combining it with the response does not substantially improve classification over response-only input.

The result is also important for interpreting RQ1. Since the response alone is highly predictive—particularly for the CNN—the strong RQ1 performance is unlikely to be explained primarily by differences in the prompts supplied to each model. Instead, the classifier appears to rely mainly on characteristics present in the generated outputs themselves.

However, this finding should be interpreted within the scope of the current dataset and model versions. It does not imply that prompts can never influence LLM identification. Certain datasets may contain systematic relationships between prompts and the models that answer them. In our controlled dataset, however, prompt-only classification provided little useful signal, while the generated responses contained substantially stronger identifying information.

5.3 RQ3: How Well Do the Models Generalize an Unseen Domain?

RQ3 evaluates whether the classifiers can identify the source of LLM when they are tested on a domain that was completely excluded from model training. Four domains were considered: **General, Code, Math, and Creative Writing**. For each experiment, one domain was removed from both the training and validation sets. The classifier was then retrained using only the remaining domains and evaluated separately on: (1) test examples from the domains seen during training and (2) test examples from the held-out domain.

This setup measures whether the LLM-identification signals learned by the classifiers generalize across different types of content rather than depending entirely on domain-specific patterns.

Table 4 Cross-domain generalization results for CNN and RNN/LSTM

Model	Held-Out Domain	Seen Accuracy	Held-Out Accuracy	Seen Macro-F1	Held-Out Macro-F1	Δ Macro-F1 (Held-Out – Seen)
CNN	General	0.9215	0.9195	0.9237	0.9196	-0.0041
CNN	Code	0.9238	0.8958	0.9236	0.8979	-0.0257
CNN	Math	0.9175	0.9236	0.9186	0.9237	+0.0052
CNN	Creative Writing	0.9315	0.8344	0.9319	0.8384	**-0.0935**
RNN/LSTM	General	0.6770	0.7845	0.6736	0.7851	**+0.1115**
RNN/LSTM	Code	0.7881	0.6042	0.7879	0.5955	**-0.1924**
RNN/LSTM	Math	0.7483	0.7197	0.7472	0.7205	-0.0267
RNN/LSTM	Creative Writing	0.7423	0.7351	0.7420	0.7283	-0.0136

The CNN showed relatively strong cross-domain generalization overall. When **General** was held out, its Macro-F1 decreased only slightly from **92.37%** on seen-domain examples to

91.96% on the held-out domain. Similarly, when **Code** was held out, Macro-F1 decreased from **92.36%** to **89.79%**. Performance on held-out **Math** was slightly higher than performance on the seen-domain subset, with Macro-F1 increasing from **91.86%** to **92.37%**.

The largest CNN generalization loss occurred for **Creative Writing**. Macro-F1 decreased from **93.19%** on seen-domain examples to **83.84%** on the held-out creative-writing examples, a reduction of approximately **9.35 percentage points**. Although performance remained well above the three-class chance level, this decline suggests that some of the CNN's learned identification cues may be less transferable to creative-writing responses when that domain is absent during training.

The RNN/LSTM exhibited greater variation across held-out domains. The largest decrease occurred when **the Code** was excluded from training. Its Macro-F1 fell from **78.79%** on seen domains to **59.55%** on held-out code, a decrease of approximately **19.24 percentage points**. Smaller reductions occurred for **Math** and **Creative Writing**, where Macro-F1 decreased by approximately **2.67** and **1.36 percentage points**, respectively.

Interestingly, the RNN/LSTM performed better on the held-out **General** subset than on its corresponding seen-domain test subset. Macro-F1 increased from **67.36%** to **78.51%**. A similar, although much smaller, effect was observed for the CNN when Math was held out. These positive differences should not be interpreted as evidence that excluding a domain from training improves generalization. The seen and held-out test subsets differ in domain composition, sample size, and difficulty, so their scores are not directly identical to test conditions.

Overall, the CNN generalized more consistently across domains than the RNN/LSTM. The CNN maintained a Macro-F1 above **83%** for every held-out domain, while the RNN/LSTM was particularly sensitive to the removal of Code. This suggests that the CNN learned LLM-associated patterns that transferred more effectively across the domains represented in this dataset, whereas the RNN/LSTM's learned representation was more dependent on the specific domain being evaluated.

At the same time, the domain-dependent changes observed for both architectures show that classification performance is not completely independent of content type. Therefore, the strong results from RQ1 should not be interpreted as reflecting only domain-invariant model fingerprints. Some of the predictive signals may interact with topic, style, formatting, or response structure, which motivates the more detailed linguistic and structural analysis in RQ4.

5.4 RQ4: What Linguistic and Structural Characteristics Help Distinguish LLM Families?

RQ4 investigates which linguistic and structural characteristics contribute to distinguishing GPT, Claude, and Gemini responses. The CNN and RNN/LSTM analyses approached this question differently. The CNN analysis examined descriptive response characteristics and local patterns associated with correctly classified examples, while the RNN/LSTM analysis used ablation experiments in which formatting, punctuation, or response length was modified. Because these analyses answer complementary aspects of RQ4, their results are reported separately rather than treated as identical experiments.

CNN Analysis of Linguistic and Structural Characteristics

The CNN full-response model used for the RQ4 analysis achieved an accuracy of **91.74%** and a Macro-F1 score of **91.82%**. To investigate possible characteristics contributing to classification, descriptive statistics were calculated from the training responses for each LLM family.

Table 5. Selected structural characteristics of training responses by LLM family

Family	Avg. Response Length (words)	Avg. Word Length	Newlines / 100 Words	Punctuation / 100 Characters	Responses with Bullet Lists	Responses with Markdown Headings
GPT	393.64	4.73	15.61	10.97	75.12%	62.90%
Claude	268.33	5.08	11.35	6.47	45.38%	20.59%

Gemini	691.92	4.81	9.66	8.72	78.33%	42.12%

The descriptive statistics show noticeable differences among the three families in this dataset. Gemini responses were substantially longer on average, with approximately **692 words per response**, compared with approximately **394 words for GPT** and **268 words for Claude**. GPT responses had the highest newline frequency and punctuation density, while GPT and Gemini used bullet lists more frequently than Claude. GPT also showed the highest frequency of Markdown headings.

These characteristics indicate that response length, formatting, and structural organization may provide useful signals for LLM-family identification. However, these statistics are descriptive associations and should not be interpreted as universal characteristics of the model families.

The CNN analysis also examined local textual patterns that contributed positively to correct classifications on the validation set. Examples included formatting-heavy patterns and phrases such as **"Here's"** for GPT, phrases such as **"Would you like me to"** for Claude, and patterns such as **"Of course"** for Gemini. These findings suggest that the CNN learned a combination of lexical and formatting cues rather than relying on a single feature. Because these patterns were identified from this specific dataset and set of model versions, they should be interpreted as supporting evidence rather than universal authorship rules.

RNN/LSTM Ablation Analysis

The RNN/LSTM analysis tested how classification performance changed when different structural cues were modified or removed.

Experiment	Accuracy	Macro Precision	Macro Recall	Macro-F1	Change in Macro-F1 vs. Full Response
Full response	**0.7600**	**0.7600**	**0.7593**	**0.7594**	—

Remove formatting	0.6853	0.6877	0.6894	0.6876	-0.0718
Remove punctuation	0.6362	0.6371	0.6389	0.6365	**-0.1229**
Response-length truncation	0.7355	0.7390	0.7386	0.7354	-0.0240

Removing formatting caused the RNN/LSTM Macro-F1 to decrease from **75.94% to 68.76%**, a reduction of approximately **7.18 percentage points**. Removing punctuation produced the largest decrease, reducing Macro-F1 to **63.65%**, approximately **12.29 percentage points below the full-response baseline**. This suggests that punctuation and formatting contain meaningful information that the RNN/LSTM uses when distinguishing the LLM families.

The response-length truncation experiment produced a smaller decrease, with Macro-F1 falling from **75.94% to 73.54%**. This experiment should be interpreted specifically as testing sensitivity to limiting response length. It does **not** fully eliminate response length as a potential confounding factor or guarantee equal length distributions across GPT, Claude, and Gemini.

Taken together, the CNN descriptive analysis and RNN/LSTM ablation results provide complementary evidence that LLM-family classification is influenced by more than semantic content alone. Response structure, formatting, punctuation, recurring local phrases, and response length all appear to contain useful predictive information in the current dataset. In particular, the relatively large RNN performance decrease after punctuation and formatting removal supports the conclusion that structural writing style contributes meaningfully to model identification.

At the same time, these results reinforce an important limitation of the overall study. Some of the strong classification performance observed in RQ1 may be driven partly by dataset-specific stylistic and structural differences rather than by stable, domain-independent properties unique to each LLM family. Consequently, the learned signals should be viewed as **detectable patterns within the selected models, domains, and dataset**, rather than permanent fingerprints that necessarily generalize all future versions of GPT, Claude, or Gemini.

6. In-Depth Analysis

6.1 CNN vs. RNN/LSTM Comparison

Across the experiments, the CNN consistently outperformed the RNN/LSTM when the generated response was available as input. The difference was already visible in RQ1, where the CNN achieved a Macro-F1 of **0.9201**, compared with **0.7594** for the RNN/LSTM.

Table 6. Summary comparison of CNN and RNN/LSTM across the main experiments

Research Question / Condition	CNN Macro-F1	RNN/LSTM Macro-F1	Main Observation
RQ1: Response-only identification	**0.9201**	0.7594	CNN substantially stronger
RQ2: Prompt only	0.3144	0.3117	Both near chance level
RQ2: Response only	**0.9281**	0.7727	CNN substantially stronger
RQ2: Prompt + Response	**0.9260**	0.7737	CNN substantially stronger
RQ3: General held out	**0.9196**	0.7851	CNN stronger on unseen General
RQ3: Code held out	**0.8979**	0.5955	Largest architecture gap
RQ3: Math held out	**0.9237**	0.7205	CNN generalizes strongly
RQ3: Creative Writing held out	**0.8384**	0.7283	Both decline, CNN remains stronger

A particularly important observation is that the difference between the two architectures appears only when useful response information is available. In the **prompt-only** RQ2 condition, both architectures performed similarly and close to chance, with Macro-F1

scores of **0.3144 for CNN** and **0.3117 for RNN/LSTM**. This suggests that the CNN advantage is not simply caused by a generally stronger classifier regardless of input. Instead, the CNN appears to be more effective at extracting predictive patterns from the generated responses.

One possible explanation is the difference in how the two architectures process text. The CNN applies convolutional filters over short windows of neighboring tokens, allowing it to detect recurring local patterns such as characteristic phrases, punctuation sequences, Markdown structures, and other short-range stylistic cues. This type of information appears particularly relevant to the current task, as the RQ4 analysis showed noticeable differences in formatting, punctuation, and recurring local expressions across the three LLM families.

The LSTM, in contrast, processes the sequence in order and summarizes information through its hidden state. This makes it suitable for learning sequential dependencies, but in this experiment, it was less effective than the CNN at distinguishing the three LLM families. One possible interpretation is that LLM-family identification in this dataset depends more heavily on repeated local stylistic and structural patterns than on long-range sequential dependencies.

The cross-domain experiments in RQ3 provide additional evidence of this difference. The CNN achieved higher held-out-domain Macro-F1 than the RNN/LSTM for **all four domains**. The difference was especially large when Code was held out: the CNN obtained a Macro-F1 of **0.8979**, whereas the RNN/LSTM obtained **0.5955**. This indicates that the CNN's learned features transferred more effectively to unseen code responses in the current dataset.

However, the results should not be interpreted as showing that CNNs are universally superior to RNNs for authorship or LLM-identification tasks. The comparison is specific to the architectures, hyperparameters, tokenizer, sequence lengths, dataset, and training setup used in this project. A different LSTM architecture, a bidirectional recurrent model, pretrained embeddings, or additional hyperparameter tuning could potentially change relative performance.

Another important consideration is that the CNN and RNN/LSTM results do not establish that either model learned a pure intrinsic “fingerprint” of each LLM family. The classifiers may use a mixture of lexical, formatting, structural, length-related, and domain-related signals. The RQ3 and RQ4 experiments show that some of these factors influence classification performance.

Overall, within the experimental setup of this project, the **CNN was the stronger and more consistent architecture for LLM-family identification**. Its advantage across response-based classification and cross-domain testing suggests that local textual and structural patterns are highly informative signals for distinguishing GPT, Claude, and Gemini responses.

6.2 Domain Generalization Analysis

RQ3 showed that LLM-family identification performance depends partly on the type of content being evaluated. Although both architectures were able to classify responses from domains that were excluded during training, the magnitude of the generalization change differed substantially across domains and architectures.

To focus specifically on generalization, Table 7 summarizes the change in Macro-F1 between the seen-domain test subset and the held-out-domain test subset. A negative value indicates lower performance on the unseen domain, while a positive value indicates that the held-out subset happened to receive a higher score than the corresponding seen-domain subset.

Table 7. Change in Macro-F1 when evaluating on held-out domains

Held-Out Domain	CNN Δ Macro-F1	RNN/LSTM Δ Macro-F1
General	-0.0041	+0.1115
Code	-0.0257	**-0.1924**
Math	+0.0052	-0.0267
Creative Writing	**-0.0935**	-0.0136

The CNN exhibited relatively stable performance when **General, Code, and Math** were held out. Its Macro-F1 changed by less than three percentage points for each of these domains. The largest CNN decrease occurred for **Creative Writing**, where Macro-F1 dropped by approximately **9.35 percentage points**, from 0.9319 on the seen-domain subset to 0.8384 on the held-out creative-writing subset.

This result suggests that some of the patterns learned by the CNN may transfer less effectively to creative-writing responses when that domain is absent from training. Creative-writing responses can differ from other domains in organization, tone, narrative structure, dialogue, and formatting. Therefore, some local lexical or structural patterns useful for distinguishing LLM families in other domains may appear differently in creative-writing text. This is a possible explanation rather than a confirmed causal finding, because the experiment did not directly isolate individual creative-writing characteristics.

The RNN/LSTM showed a different pattern. Its largest generalization loss occurred when **the Code** was held out. Macro-F1 decreased from 0.7879 on the seen-domain test subset to 0.5955 on held-out Code, corresponding to a decrease of approximately **19.24 percentage points**. This was the largest generalization loss observed for either architecture.

One possible interpretation is that code responses contain structural patterns that differ considerably from ordinary natural-language responses, including programming syntax, indentation, code blocks, technical explanation styles, and specialized tokens. If the RNN/LSTM was not exposed to code during training, it may have had greater difficulty transferring the sequential patterns learned from General, Math, and Creative Writing to this structurally different domain. Again, this should be treated as a hypothesis supported by the observed performance pattern rather than as a directly proven cause.

The Math domain produced comparatively stable results for both architectures. The CNN's held-out Math Macro-F1 was slightly higher than its seen-domain score, while the RNN/LSTM showed only a modest reduction of approximately **2.67 percentage points**. This suggests that, within the current dataset, some of the model-family signals learned from the other domains transferred reasonably well to mathematical responses.

An unusual result occurred when **General** was held out for the RNN/LSTM. Its held-out Macro-F1 of 0.7851 was higher than its seen-domain Macro-F1 of 0.6736. This should not be interpreted as evidence that withholding General from training improves the model. The held-out and seen-domain evaluations contain different examples and different mixtures of domains, so they can differ in intrinsic difficulty. In particular, the seen-domain subset in this experiment consists of Code, Math, and Creative Writing examples, whereas the held-out subset contains only General examples. Therefore, the positive difference may reflect differences in subset difficulty rather than a true benefit from domain exclusion.

The domain-generalization results also strengthen an important conclusion from RQ1. High classification accuracy does not necessarily mean that the classifiers learned a single domain-independent fingerprint for each LLM. If the identifying signal were completely independent of content domain, performance would be expected to remain nearly unchanged across all held-out-domain experiments. Instead, the observed drops for CNN on Creative Writing and RNN/LSTM on Code indicate that at least some predictive cues interact with domain-specific characteristics.

At the same time, the CNN's relatively strong held-out performance demonstrates that its learned signals were not completely tied to the training domains. Even in its weakest held-out condition, Creative Writing, the CNN retained a Macro-F1 of **0.8384**. This suggests that the CNN learned a mixture of domain-specific and more transferable LLM-associated patterns.

Overall, the domain analysis indicates that **LLM-family identification generalizes beyond the domains seen during training, but the strength of that generalization depends on both the architecture and the held-out domain**. The CNN was generally more robust across domain shifts, while the RNN/LSTM was especially sensitive to unseen code.

6.3 Linguistic and Structural Signal Analysis

RQ4 provides evidence that the classifiers rely not only on the semantic content of responses, but also on **linguistic and structural characteristics** of the generated text. The CNN analysis and the RNN/LSTM ablation experiments approach this question differently and therefore provide complementary evidence.

The CNN analysis revealed noticeable family-level differences in response structure within the training data. Gemini produced the longest responses on average, with approximately **691.92 words per response**, compared with **393.64 words for GPT** and **268.33 words for Claude**. GPT showed the highest punctuation density and newline frequency, while bullet-list usage was particularly common for GPT and Gemini. Markdown headings also appeared substantially more often in GPT responses than in Claude responses.

Table 8. Selected structural characteristics observed in the CNN analysis

Characteristic	GPT	Claude	Gemini
Average response length (words)	393.64	268.33	**691.92**
Average word length	4.73	**5.08**	4.81
Newlines per 100 words	**15.61**	11.35	9.66
Punctuation per 100 characters	**10.97**	6.47	8.72

Responses containing bullet lists	75.12%	45.38%	**78.33%**
Responses containing Markdown headings	**62.90%**	20.59%	42.12%

These differences demonstrate that the three LLM families in the curated dataset do not produce identically structured responses. However, the descriptive statistics alone do not prove that the CNN directly relied on each of these characteristics. They instead identify plausible sources of predictive signal that are consistent with the classifier's strong performance.

The CNN's local-pattern analysis provides additional evidence that short lexical and stylistic patterns may contribute to classification. Examples associated with correctly classified responses included phrases such as **“Here’s”** for GPT, **“Would you like me to”** for Claude, and **“Of course”** for Gemini. Such patterns are particularly relevant to a CNN because convolutional filters are designed to detect recurring local token combinations.

These examples should not be interpreted as deterministic rules. A phrase such as “Of course” is not inherently unique to Gemini, nor is “Here’s” unique to GPT. Instead, their usefulness depends on how frequently and in what contexts such patterns appeared in the dataset used for training.

Evidence from RNN/LSTM Ablations

The RNN/LSTM experiments provide a more direct test of whether certain structural cues contribute to classification. Performance was measured after modifying the responses and compared against the full-response baseline.

Table 9. Effect of structural modifications on RNN/LSTM performance

RQ4 Experiment	Macro-F1	Change from Full Response
Full response	**0.7594**	—
Remove formatting	0.6876	-0.0718
Remove punctuation	0.6365	**-0.1229**
Response-length truncation	0.7354	-0.0240

Removing formatting reduced Macro-F1 by approximately **7.18 percentage points**, indicating that structural organization such as newlines and Markdown-related formatting contained useful predictive information.

The largest reduction occurred when punctuation was removed. Macro-F1 fell from **0.7594 to 0.6365**, a decrease of approximately **12.29 percentage points**. This suggests that punctuation patterns provide substantial information for distinguishing the LLM families in this dataset. Differences in the use of commas, colons, quotation marks, parentheses, Markdown symbols, and other punctuation may therefore form part of the learned stylistic signal.

The response-length truncation experiment caused a smaller reduction of approximately **2.40 percentage points**, from 0.7594 to 0.7354. This indicates that restricting available response length affects classification performance, but the effect was smaller than removing formatting or punctuation.

Importantly, the truncation experiment does **not** establish that response length is unimportant. The three families showed large differences in their original response-length distributions, particularly Gemini's substantially longer average responses. The truncation experiment limits the amount of text available to the model, but it does not create perfectly matched response-length distributions across families. Therefore, response length remains a potential source of classification signal and a possible confounding factor.

Combining the CNN and RNN/LSTM Evidence

Taken together, the two analyses suggest that LLM-family identification in this project is based on a **combination of signals rather than a single feature**. These signals may include:

- recurring local phrases and lexical choices,
- punctuation patterns,
- Markdown and formatting behavior,
- response organization,
- use of bullet lists and headings,
- response length,
- and potentially interactions between these characteristics.

The RNN/LSTM ablations are particularly useful because classification performance declined when some structural information was removed. At the same time, the CNN's stronger overall performance suggests that local combinations of lexical and structural cues may be especially informative for this task.

The findings also help explain why the term **“fingerprint” should be used cautiously**. The classifiers are not necessarily detecting an intrinsic property that permanently identifies a model

family. Instead, they appear to exploit observable regularities in how the selected versions of GPT, Claude, and Gemini generated responses in this dataset. These regularities may change with new model versions, different prompting strategies, different system instructions, or different domains.

6.4 Error and Confusion Analysis

The confusion matrices from RQ1 provide additional insight into **which LLM families were most difficult to distinguish** and how the error patterns differed between the CNN and RNN/LSTM. Rather than considering only overall accuracy, this analysis examines where the classifiers made mistakes.

Table 10. RQ1 class-level classification performance

Actual Family	Test Samples	CNN Correct	CNN Recall	RNN/LSTM Correct	RNN/LSTM Recall
GPT	273	259	**94.87%**	208	76.19%
Claude	295	266	**90.17%**	217	73.56%
Gemini	328	299	**91.16%**	256	78.05%

The CNN performed strongly across all three families. GPT had the highest CNN recall at approximately **94.87%**, while Claude had the lowest at approximately **90.17%**. Although Claude was somewhat more difficult for the CNN to classify, the difference among the three families was relatively small.

The RNN/LSTM exhibited substantially more classification errors. Its recall was approximately **76.19% for GPT**, **73.56% for Claude**, and **78.05% for Gemini**. Claude again had the lowest recall, although all three classes showed considerably more confusion than under the CNN.

Which Families Were Confused?

For the CNN, the most frequent directional error was:

- **Claude → Gemini:** 20 responses

Other notable CNN errors included:

- Gemini → Claude: 17
- Gemini → GPT: 12
- GPT → Claude: 9
- Claude → GPT: 9
- GPT → Gemini: 5

The strongest pairwise confusion therefore occurred between **Claude and Gemini**. Together, 37 responses were confused between these two families in either direction.

The RNN/LSTM showed a broader pattern of errors. Its largest directional confusion was also:

- **Claude → Gemini:** 46 responses

Other relatively common RNN/LSTM errors included:

- Gemini → GPT: 41
- GPT → Gemini: 34
- Claude → GPT: 32
- GPT → Claude: 31
- Gemini → Claude: 31

This suggests that the LSTM representation produced less clearly separated family boundaries than the CNN representation in the current experiment.

Interpreting the Error Patterns

One possible explanation for the Claude–Gemini confusion is that some responses from these families may share lexical, structural, or formatting characteristics in the curated dataset. However, the confusion matrix alone cannot establish why a particular response was misclassified. Therefore, the observed errors should not be interpreted as proof that Claude and Gemini generally have similar writing styles.

The RQ4 results provide some useful context. The families differed in characteristics such as response length, punctuation, formatting, and use of structural elements. At the same time, these characteristics can overlap across individual responses. A Claude response containing formatting or phrasing more commonly associated with Gemini in the training

data, for example, could potentially be assigned to the wrong class. This is a plausible interpretation rather than a directly verified explanation for individual errors.

Another important observation is that **Gemini had the largest number of test examples (328)**, followed by Claude (295) and GPT (273). For this reason, raw error counts alone should not be used to determine which family was most difficult. Class-level recall provides a more appropriate comparison because it accounts for the number of examples in each family. Based on recall, Claude was the most difficult of the three families for both architectures in RQ1.

The difference between the CNN and RNN/LSTM confusion matrices also supports the broader architectural findings from Section 6.1. The CNN produced a much stronger diagonal concentration, indicating that most examples were assigned to their correct families. The RNN/LSTM showed greater dispersion across incorrect classes, consistent with its lower overall Macro-F1.

Error Analysis Limitation

The current analysis examines errors at the **family level**, but it does not manually inspect every misclassified response. Therefore, we cannot determine with certainty whether individual mistakes were caused by response length, formatting, domain, lexical choices, specific model versions, or combinations of these factors.

A more detailed future study could manually inspect misclassified responses or use feature attribution methods to determine which tokens and structural patterns contributed most strongly to individual predictions.

6.5 Ablation Analysis

The ablation-style experiments help determine whether classification performance depends on particular sources of information rather than on the full text alone. In this project, two types of evidence are especially useful: the RQ2 comparison of prompt and response information, and the RNN/LSTM RQ4 experiments that removed or restricted structural information.

Table 11. Summary of information-removal and modification experiments

Experiment	Model	Macro-F1	Change from Corresponding Full/Baseline Input
RQ2: Response only	CNN	**0.9281**	Baseline
RQ2: Prompt only	CNN	0.3144	-0.6137
RQ2: Prompt + Response	CNN	0.9260	-0.0021
RQ2: Response only	RNN/LSTM	**0.7727**	Baseline
RQ2: Prompt only	RNN/LSTM	0.3117	-0.4610
RQ2: Prompt + Response	RNN/LSTM	0.7737	+0.0010
RQ4: Full response	RNN/LSTM	**0.7594**	Baseline
RQ4: Remove formatting	RNN/LSTM	0.6876	-0.0718
RQ4: Remove punctuation	RNN/LSTM	0.6365	**-0.1229**
RQ4: Response-length truncation	RNN/LSTM	0.7354	-0.0240

The RQ2 experiments provide the clearest evidence about where the primary classification signal originates. When only the user prompt was available, Macro-F1 dropped to **0.3144 for CNN** and **0.3117 for RNN/LSTM**, compared with response-only Macro-F1 scores of **0.9281** and **0.7727**, respectively. Because this is a three-class classification problem, the weak prompt-only performance indicates that the prompt provides little useful information for identifying the responding LLM family in this dataset.

Adding the prompt back to the generated response produced almost no benefit. For the CNN, Macro-F1 changed slightly from **0.9281 to 0.9260**, while for the RNN/LSTM it changed from **0.7727 to 0.7737**. This provides additional evidence that the generated response itself contains nearly all of the useful identifying information captured by these classifiers.

The RNN/LSTM RQ4 experiments further show that not all information within the response contributes equally. Removing formatting reduced Macro-F1 from **0.7594 to 0.6876**, while removing punctuation caused a larger reduction to **0.6365**. The stronger effect from

punctuation removal suggests that punctuation-related patterns form an important part of the signal learned by the recurrent classifier.

Response-length truncation produced a smaller reduction in Macro-F1, from **0.7594 to 0.7354**. This indicates that limiting the available amount of text weakens classification performance, but less severely than removing punctuation or formatting in this experiment. However, this experiment does not completely control for response length across the three LLM families and therefore should not be interpreted as proving that length is only a minor factor.

Taken together, the ablation results indicate that the classifiers are not merely associating the user prompt with a particular LLM. Instead, most of the identifying information resides within the generated response itself. Within those responses, structural characteristics—particularly punctuation and formatting—contribute meaningful predictive information.

The results also support the broader conclusion that LLM-family identification is based on a **combination of cues** rather than a single isolated feature. Removing one type of information reduces performance but does not eliminate classification ability entirely, suggesting that lexical choices, local phrases, structure, punctuation, formatting, and response length may collectively contribute to the learned representation.

7. Lessons and Experience

7.1 What worked and what didn't

Several aspects of the project worked well. The controlled dataset design, fixed train/validation/test splits, and training-only vocabulary construction allowed the CNN and RNN/LSTM models to be compared under a consistent setup. Grouping identical prompts into the same split also helped reduce the risk of prompt leakage.

The CNN performed particularly well for LLM-family identification. It achieved approximately **92% accuracy** in RQ1 and remained relatively strong across the held-out-domain experiments. The RQ2 experiments also worked well for understanding where the predictive signal came from: prompt-only classification performed near chance, while response-based classification was substantially stronger.

The RQ4 experiments were also useful because they showed that formatting, punctuation, response structure, and local textual patterns contributed to classification.

Not everything worked equally well. The RNN/LSTM consistently performed below the CNN in response-based classification, and its performance was particularly sensitive when Code was held out during RQ3. In addition, the response-length experiment did not completely eliminate length differences among the three LLM families. Therefore, it was interpreted as a **response-length truncation experiment** rather than a true length-matching experiment.

Overall, the project showed that strong predictive performance is useful, but additional experiments are necessary to understand what information the classifiers actually use.

7.2 Challenges and how they were resolved

One challenge was preparing the original LMArena data for a controlled classification task. The source dataset contained different model versions, repeated prompts, multiple domains, and an unequal number of examples across LLM families. This was addressed by selecting representative models, filtering the data, grouping identical prompts, balancing the three target families, and constructing fixed train, validation, and test splits.

Another challenge was preventing information leakage. Because the same prompt could appear in multiple records, randomly splitting individual rows could place identical prompts into different dataset partitions. To reduce this risk, identical prompts were grouped using a prompt-based identifier so that all examples associated with the same exact prompt remained in the same split.

For the RNN/LSTM, a challenge was converting text into numerical sequences of different lengths. We used tokenization, padding, and actual sequence lengths to prepare the inputs. Packed sequences were then used so that padding tokens would not affect the LSTM's final hidden representation.

Another challenge was maintaining consistency between the CNN and RNN/LSTM experiments. Since different team members worked on the two architectures, the models initially differed in some preprocessing and evaluation details. These were resolved by

using the same fixed data splits, consistent input settings for RQ2, training-only vocabularies, common evaluation metrics, and validation-based checkpoint selection.

Finally, organizing multiple RQ experiments and result files requires careful coordination. Consistent folder organization, experiment naming, and saving final outputs helped distinguish final results from earlier runs.

7.3 What we would do differently

If we repeated the project, we would perform more extensive hyperparameter tuning for both architectures earlier in the development process. For the RNN/LSTM specifically, we would test different sequence lengths, hidden dimensions, dropout rates, and other model settings earlier rather than relying mainly on one configuration.

We would also spend more time comparing the RNN/LSTM's performance across input types and held-out domains to better understand why it struggled more than the CNN in certain settings, particularly when Code was excluded from training.

For the dataset, we would attempt to control response-length distributions more directly across GPT, Claude, and Gemini. The current dataset contains noticeable differences in average response length, which may provide an additional classification signal. A future experiment could construct explicitly length-matched subsets to isolate the effect of response length more carefully.

We would also investigate semantic near-duplicate prompts rather than grouping only exact prompt matches. The current splitting procedure prevents exact prompt overlap, but semantically similar prompts may still appear in different splits.

Finally, we would perform more detailed analysis of misclassified responses. Manual error inspection or feature-attribution methods could help determine whether incorrect predictions are caused primarily by lexical choices, formatting, domain, response length, model version, or combinations of these factors.

8. Conclusion and Limitations

8.1 Conclusion

This project investigated whether generated text contains enough information to identify the LLM family that produced it. Using a curated dataset of **6,000 responses from GPT, Claude, and Gemini**, we compared CNN and RNN/LSTM classifiers and examined four research questions covering overall identification performance, the contribution of user prompts, cross-domain generalization, and linguistic or structural signals.

The results show that LLM-family identification is feasible within the experimental setting of this project. For RQ1, the CNN achieved **91.96% accuracy and 92.01% Macro-F1**, substantially outperforming the RNN/LSTM, which achieved **76.00% accuracy and 75.94% Macro-F1**. This indicates that the generated responses contain detectable patterns associated with their source model families.

RQ2 showed that these identifying signals originate primarily from the **generated responses rather than the user's prompts**. Prompt-only classification performed close to chance for both architectures, while response-only classification was substantially stronger. Combining the prompt with the response provided almost no additional benefit. This suggests that the classifiers were not simply learning associations between particular prompts and model labels.

RQ3 demonstrated that some of the learned signals generalized across content domains, although performance depended on both the architecture and the held-out domain. The CNN was generally more stable across General, Code, Math, and Creative Writing. The RNN/LSTM showed greater domain sensitivity, with its largest decrease occurring when **Code** was excluded from training.

RQ4 provided further evidence that classification depends on multiple linguistic and structural characteristics. The CNN analysis revealed differences in response length, formatting, punctuation, organization, and recurring local phrases across the three model families. The RNN/LSTM ablation experiments showed noticeable performance reductions when formatting and punctuation were removed, with punctuation removal producing the largest decrease.

Overall, the results suggest that GPT, Claude, and Gemini responses contain distinguishable patterns within the selected dataset and model versions. The CNN was the

stronger architecture in this project, indicating that **local lexical and structural patterns were particularly informative for LLM-family identification**.

However, these patterns should not be interpreted as permanent or universal model fingerprints. The classifiers likely rely on a combination of stylistic, structural, lexical, domain-related, and length-related signals that may change as models, prompting strategies, and datasets change.

8.2 Limitations

Several limitations should be considered when interpreting the results.

First, the study focuses on **three LLM families—GPT, Claude, and Gemini—and a selected set of model versions within each family**. Therefore, the results may not generalize every version of these models or other LLM families.

Second, the final dataset was intentionally balanced to contain **2,000 examples per family** and used a controlled domain distribution. This design improves comparability between classes, but it does not represent the natural distribution of responses in the original LMArena dataset. Consequently, the reported performance reflects the controlled experimental dataset rather than real-world model usage frequencies.

Third, substantial differences remained in **response length** across the three families. Gemini responses, for example, were considerably longer on average than GPT and Claude responses. Although the RQ4 response-length truncation experiment examined sensitivity to limiting available text, it did not create perfectly matched length distributions. Response length may therefore remain a confounding signal.

Fourth, the data split grouped **exactly identical prompts** to prevent them from appearing across train, validation, and test sets. However, semantically similar prompts with different wording may still occur in different splits. A stronger future analysis could use semantic similarity methods to identify and group near-duplicate prompts.

Fifth, the dataset contains multiple model versions within each family. Although representative high-frequency models were selected, the number of examples from each version was not perfectly equal. The classifier may therefore learn some signals associated with individual model versions in addition to broader family-level characteristics.

Sixth, the RQ3 domain labels are based on the available dataset metadata. The **General** category represents responses not labeled as Code, Math, or Creative Writing rather than a

single homogeneous topic. This makes interpretation of the General category broader than the other three domains.

Seventh, the CNN and RNN/LSTM experiments used specific architectural and hyperparameter choices. The stronger CNN performance does not establish that CNNs are universally superior to recurrent models for LLM attribution. More extensive hyperparameter tuning, bidirectional recurrent architectures, pretrained representations, or other classifiers could produce different results.

Finally, the current analysis identifies correlations between textual characteristics and model families but does not establish that any particular feature is a causal or permanent model signature. Model outputs can change because of new model releases, system prompts, decoding settings, fine-tuning, or other generation conditions.

8.3 Future Work

Future work could extend this project by including additional LLM families and newer model versions, constructing more strictly length-controlled datasets, and grouping semantically similar prompts during data splitting. Additional experiments could also investigate whether a classifier trained on one generation of an LLM family continues to identify newer versions of that family.

More detailed error analysis could examine individual misclassified responses and use feature-attribution techniques to determine which tokens or structural characteristics contributed most strongly to predictions. Finally, comparing CNN and RNN/LSTM models with transformer-based classifiers could provide a broader understanding of which architectures are most effective for LLM-family attribution.

References

LMArena. (2025). *Arena Human Preference 140K* [Data set]. Hugging Face. Hugging Face

Kim, Y. (2014). Convolutional neural networks for sentence classification. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1746–1751. https://doi.org/10.3115/v1/D14-1181 ACL Anthology

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735 MIT Press Direct

Appendix

GitHub Repository: https://github.com/saiyadauroosha/llm-fingerprint-classification

A. Team member contributions

- **Saiyada Uroosha Shahnawaz — Team Lead:** Led project coordination and integration. Curated and cleaned the dataset, selected the final GPT/Claude/Gemini model families and domains, created the fixed train/validation/test splits, organized the GitHub repository, coordinated CNN and RNN experiments, reviewed final results, and contributed to the final report and cross-model analysis.
- **Vanisha Gupta-**Developed the CNN architecture and worked on the CNN experiments for RQ1, RQ3, and RQ4. Contributed to Section 3 and parts of Section 4 of the final report, including descriptions of the CNN architecture, model implementation, and training details. Coordinated with Ariana Sookoo to work on the CNN architecture.
- **Ariana Sookoo —** Created a base CNN file for project and refined the CNN experiments for RQ2 once developed by partner. Created the results files for CNN experiments. Coordinated with Vanisha Gupta to ensure proper version control of files to the GitHub repository.
- **Erin Jibu -** Worked on RQ2 and RQ3 and also aided Ananya in RQ1 and RQ4. Contributed to the implementation and evaluation of the RNN models. Worked with Ananya with testing and analysis of the results. Pushed the results file to the shared GitHub repository.
- **Ananya Muppirala-** Worked mainly on RQ1 and RQ4 but also contributed to helping Erin when needed with RQ2 and RQ3. Contributed to model implementation, results tracking, and analysis as well.

B. Statement on LLM assistance

Generative AI tools, primarily ChatGPT, were used during this project as a support resource. They were used to clarify machine-learning concepts, review and troubleshoot parts of the CNN and RNN/LSTM implementations, organize the experimental workflow, interpret evaluation results, and improve the clarity and structure of the project report.

All dataset preparation, model training, experiments, and reported numerical results were produced from the team's own code and project data. AI-generated suggestions were reviewed and verified before being incorporated into the project. The team remained responsible for understanding the methods, validating the results, and making the final decisions regarding the implementation, analysis, and conclusions presented in this report.